Lab no: 11 **Date:2082/03/09**

Title: Implementation of Standard Date and Time format specifiers in C#.

Theory:

C# provides a rich set of standard date and time format specifiers that allow you to format DateTime values easily without writing custom code. Each specifier is a single character, used with ToString() or string interpolation to display a formatted date and time string.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DateTimeFormatSpecification
{
    internal class Program
    {
        static void Main(string[] args)
        {
            DateTime dt=DateTime.Now;
            //Console.WriteLine("{0:d}",dt);
            //Console.WriteLine("{0}",dt);
            //Console.WriteLine("{0:D}",dt};
            //Console.WriteLine("{0:f}",dt};
            //Console.WriteLine("{0:F}",dt};
            //Console.WriteLine("{0:g}",dt};
            //Two or more specifications
            Console.WriteLine("{0:d} {1:D}",dt,dt);
            //Console.WriteLine("{0:F}", dt);
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu
10/30/2025


C:\Users\Acer\source\repos\DateTimeFormatSpecification\DateTimeFormatSpecification\bin\Debug\DateTimeFormatSpecification
.exe (process 12244) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window
```

```
10/30/2025 12:02:31 PM


C:\Users\Acer\source\repos\DateTimeFormatSpecification\DateTimeFormatSpecification\bin\Debug\DateTimeFormatSpecification
.exe (process 16376) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
10/30/2025 12:02:55 PM
```

```
10/30/2025 12:03:30 PM


C:\Users\Acer\source\repos\DateTimeFormatSpecification\DateTimeFormatSpecification\bin\Debug\DateTimeFormatSpecification
.exe (process 21904) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
Thursday, October 30, 2025 12:08:15 PM


C:\Users\Acer\source\repos\DateTimeFormatSpecification\DateTimeFormatSpecification\bin\Debug\DateTimeFormatSpecification
.exe (process 18676) exited with code 0 (0x0).
Press any key to close this window . . .
```

```
10/30/2025 Thursday, October 30, 2025



C:\Users\Acer\source\repos\DateTimeFormatSpecification\DateTimeFormatSpecification\bin\Debug\DateTimeFormatSpecification
.exe (process 10020) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Lab no: 12 **Date:2082/03/10**

Title: (a) Implementation of Arithmetic Operators using System in C#.

Theory:

In C#, arithmetic operators let you perform basic math on numbers, like addition (+), subtraction (–), multiplication (*), division (/), and modulus (remainder, %). You also have increment (++) and decrement (--) to adjust values by 1. These operators work with all numeric types and follow standard math rules, including operator precedence.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab12a
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 10, b = 5, c, d, e, f, g;
            c = a + b;
            d = a - b;
            e = a / b;
            f = a * b;
            g = a % b;
            Console.WriteLine(c);
            Console.WriteLine(d);
            Console.WriteLine(e);
            Console.WriteLine(f);
            Console.WriteLine(g);
            Console.ReadLine();
        }
    }
}
```

Output:

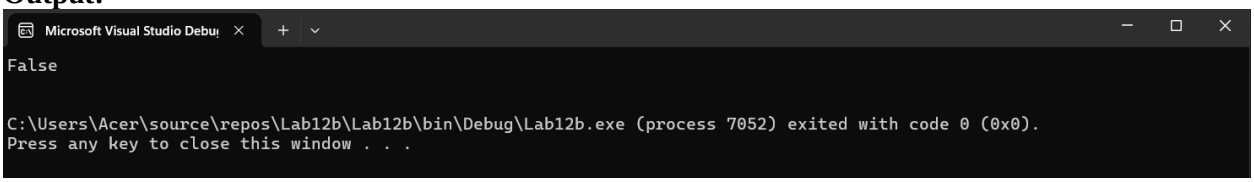
(b) Implementation of Logical Operators(Equal) using System in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab12b
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 20;
            int b = 50;
            bool c = a == b;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:



(c) Implementation of Logical Operators (Not Equal) using System in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace LAb12c
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 10;
            int b = 20;
            bool c = a != b;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

True


C:\Users\Acer\source\repos\LAb12c\LAb12c\bin\Debug\LAb12c.exe (process 16376) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

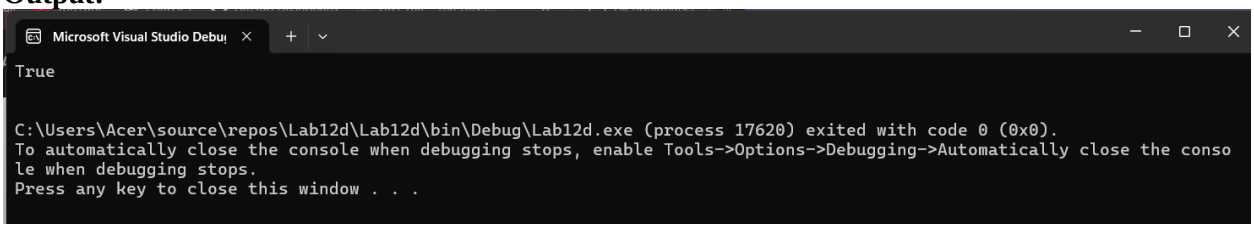
(d) Implementation of Logical Operators (Greater than) using System in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab12d
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            int b = 10;
            bool c = a > b;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:



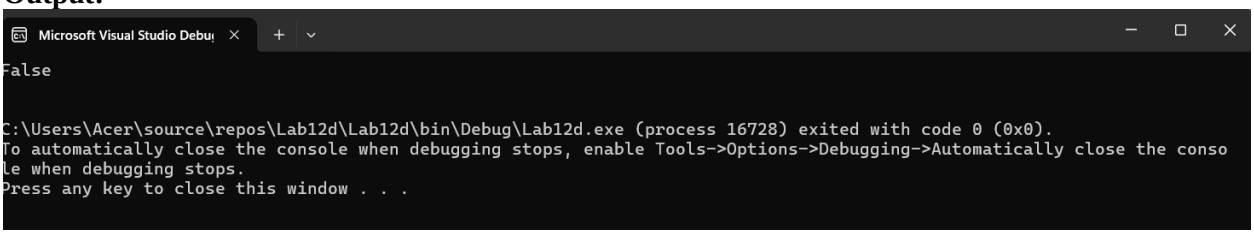
(e) Implementation of Logical Operators (Less than) using System in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab12e
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            int b = 10;
            bool c = a < b;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:



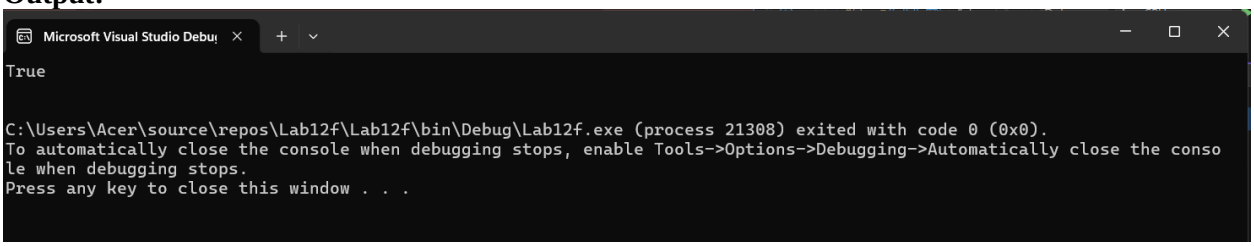
(f) Implementation of Logical Operators (Greater than or Equal to) using System in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab12f
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            int b = 10;
            bool c = a >= b;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:



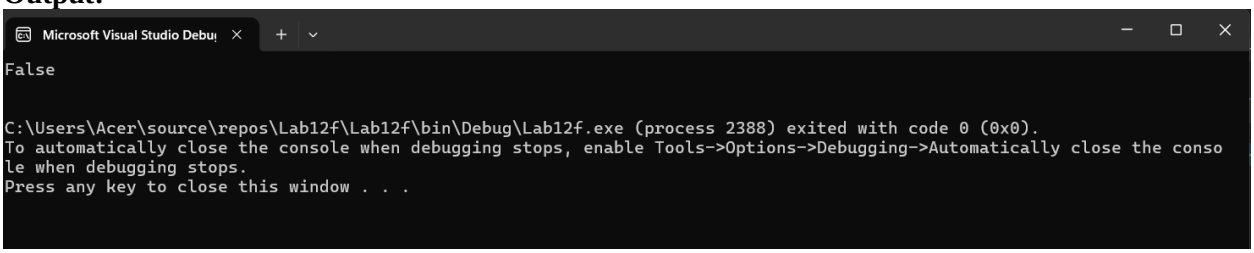
(g) Implementation of Logical Operators (Less than or Equal to) using System in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Lab12f
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 15;
            int b = 10;
            bool c = a <= b;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:



Lab no: 13 **Date:2082/03/11**

Title: (a) Implementation of Logical and Conditional Operator (AND or &&) using System in C#.

Theory:

In C#, the logical AND (&) and conditional AND (&&) operators are used to check multiple conditions.

- & (Logical AND): Evaluates both conditions always.
- && (Conditional AND): Uses short-circuiting, meaning the second condition is checked only if the first is true.

Both return true only when all conditions are true, and are commonly used in decision-making statements like if or while.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _13a.Conditional_operator__AND_
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 40;
            int b = 20;
            bool c = (a > b) && (b < a) ;
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

True


C:\Users\Acer\source\repos\_13a.Conditional_operator__AND_\_13a.Conditional_operator__AND_\bin\Debug\_13a.Conditional_op
erator__AND_.exe (process 20652) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

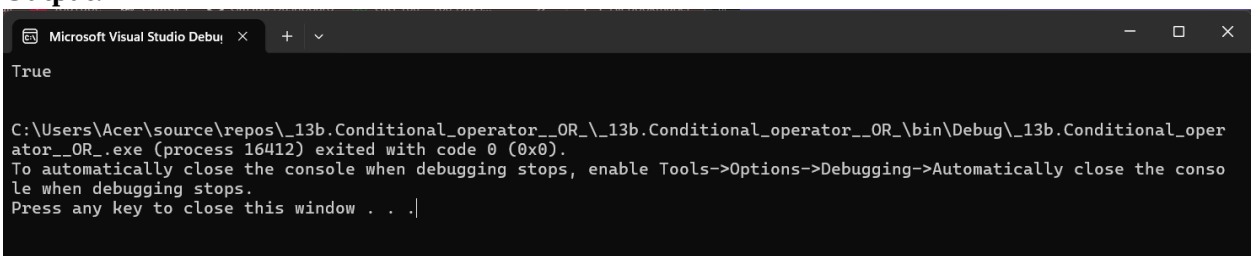
b. Implementation of Logical and Conditional Operator (OR or ||) using System in C#.

Source code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _13b.Conditional_operator__OR_
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 40;
            int b = 20;
            bool c = (a >b) || (b < a);
            Console.WriteLine(c);
            Console.ReadLine();
        }
    }
}
```

Output:



Lab no: 14 **Date:2082/03/11**

Title: (a) Implementation of Compound Assignment Operator using System in C#.

Theory:

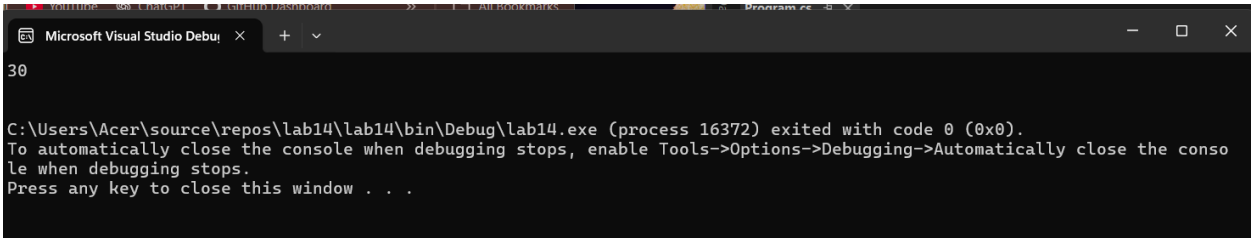
In C#, compound assignment operators perform an operation and assignment in one step, simplifying code. Examples include +=, -=, *=, /=, and %=. For instance, x += 5 means x = x + 5. These operators make expressions shorter and easier to read.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace lab14
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 20;
            a += 10;
            Console.WriteLine(a);
            Console.ReadLine();
        }
    }
}
```

Output:



b.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace lab14
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 60;
            a /= 3;
            Console.WriteLine(a);
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

20


C:\Users\Acer\source\repos\lab14\lab14\bin\Debug\lab14.exe (process 21480) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Lab no: 15 **Date:2082/03/11**

Title: Implementation of Ternary Operator using System in C#.

Theory:

In C#, the ternary operator (?:) is a shorthand for an if-else statement. It checks a condition and returns one of two values based on whether the condition is true or false.

Example: result = (a > b) ? a : b; — this assigns the larger of a and b to result.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TernaryOperator
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter the age of person");
            int a=int .Parse(Console.ReadLine());
            string b = (a >= 18) ? "You are eligible to vote" : "You  are not
eligible to vote";
            Console.WriteLine(b);
            Console.ReadLine();

        }
    }
}
```

Output:

```
C:\Users\Acer\source\repos\T
Enter the age of person
22
You are eligible to vote
```

```
Microsoft Visual Studio Debu
Enter the age of person
12
You  are not eligible to vote


C:\Users\Acer\source\repos\TernaryOperator\TernaryOperator\bin\Debug\TernaryOperator.exe (process 10412) exited with cod
e 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Lab no: 16 **Date:2082/03/12**

Title: Implementation of Precedence Operator using System in C#.

Theory:

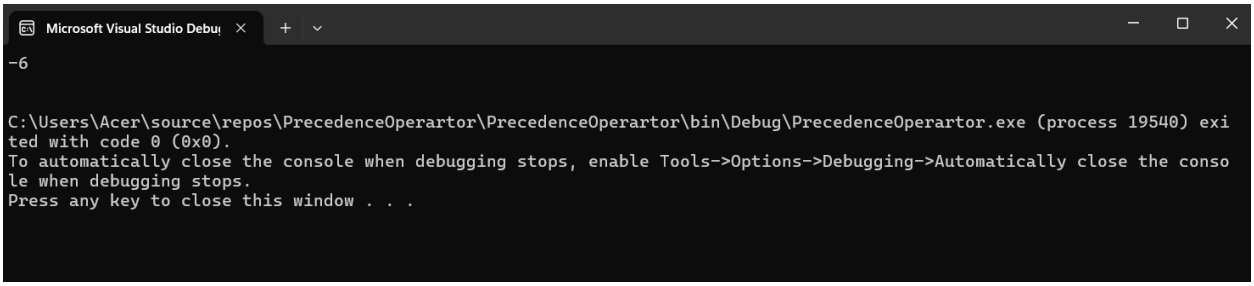
In C#, operator precedence determines the order in which operators are evaluated in an expression. Operators with higher precedence are executed before those with lower precedence. For example, in int result = 10 + 5 * 2;, multiplication (*) is performed before addition (+), giving result = 20. Parentheses () can be used to change the evaluation order.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PrecedenceOperartor
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a = 8 / 2 - (3 + 2) * 2;
            Console.WriteLine(a);
            Console.ReadLine();
        }
    }
}
```

Output:



Lab no: 17 **Date:2082/03/12**

Title:(a) Implementation of Default Construction in C#.

Theory:

In C#, a constructor is a special method that is automatically called when an object of a class is created. Its name is the same as the class name and it has no return type.

A default constructor is a constructor that takes no parameters. It is used to initialize objects with default values or display a message when an object is created.

In this program, the default constructor Program() is executed automatically when the object program is created inside the Main() method, displaying messages on the console.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConstructorisInCsharpLab1
{
    internal class Program
    {


        //creating default constructors with same name program
        public Program() {
            Console.WriteLine("Default Constructor for lab in Net Centric");
            Console.WriteLine("This is MMBMCSIT Lab oon default constructor");
        }
        static void Main(string[] args)
        {
            Program program = new Program();
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

Default Constructor for lab in Net Centric
This is MMBMCSIT Lab oon default constructor


C:\Users\Acer\source\repos\ConstructorisInCsharpLab1\ConstructorisInCsharpLab1\bin\Debug\ConstructorisInCsharpLab1.exe (
process 20484) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

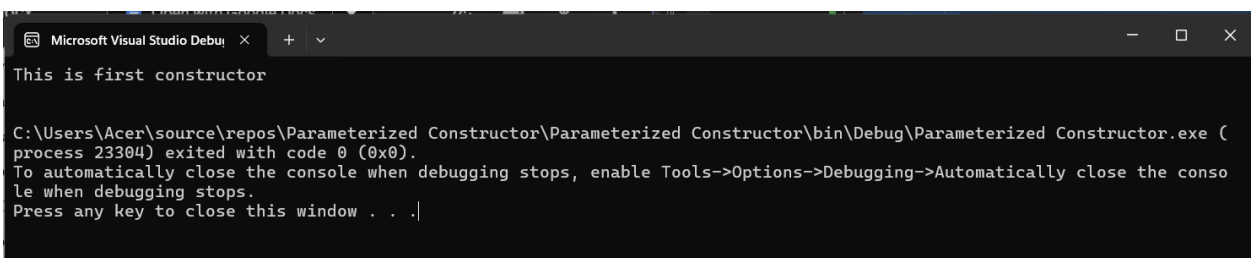
(b) Implementation of Parameterized Construction in C#

1.Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Parameterized_Constructor
{
    internal class Program
    {
        public Program()
        {
            Console.WriteLine("This is first constructor");
        }
        public Program(int a, int b)
        {
            Console.WriteLine("This is second constructor:{0}", (a + b));
        }
        public Program(int a, int b, int c)
        {
            Console.WriteLine("This id third constructor:{0}", (a + b + c));
        }
        static void Main(string[] args)
        {
            Program p = new Program();
            Console.ReadLine();
        }
    }
}
```

Output:



2.Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Parameterized_Constructor
{
    internal class Program
    {
        public Program()
        {
            Console.WriteLine("This is first constructor");
        }
        public Program(int a, int b)
        {
            Console.WriteLine("This is second constructor:{0}", (a + b));
        }
        public Program(int a, int b, int c)
        {
            Console.WriteLine("This id third constructor:{0}", (a + b + c));
        }
        static void Main(string[] args)
        {
            Program p = new Program(35,8);
            Console.ReadLine();
        }
    }
}
```

Output:

3.Source code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Parameterized_Constructor
{
    internal class Program
    {
        public Program()
        {
            Console.WriteLine("This is first constructor");
        }
        public Program(int a, int b)
        {
            Console.WriteLine("This is second constructor:{0}", (a + b));
        }
        public Program(int a, int b, int c)
        {
            Console.WriteLine("This id third constructor:{0}", (a + b + c));
        }
        static void Main(string[] args)
        {
            Program p = new Program(35,8,45);
            Console.ReadLine();
        }
    }
}
```

Output:

(c) Implementation of Copy Construction in C#

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CopyConstructor
{

    class Example
    {
        string name;
        int age;
        public Example(string name, int age)
        {
            this.name = name;
            this.age = age;
        }
        public Example(Example e)//copy cintructor
        {
            this.name = e.name;
            this.age = e.age;
        }
        public void GetData()
        {
            Console.WriteLine("Name is:{0}", name);
            Console.WriteLine("Age is:{0}", age);

        }
    }
        internal class Program
        {
            static void Main(string[] args)
        {
            Example Obj = new Example("Adit",25);
            Obj.GetData();
            Example Obj1= new Example(Obj);
            Obj1.GetData();
            Console.ReadLine();
        }
      }
}
```

Output:

(d) Implementation of Private Construction in C#

Source Code:

```
using PrivateConstructor;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Threading.Tasks;

namespace PrivateConstructor
{
    class Example
    {
        public static int a;
        private Example()
        {
        }
         public static int getIncrement()
        {
            return ++a;
        }
        public static void GetTime()
        {
            Console.WriteLine(DateTime.Now);
        }

        }
    }
    internal class Program
    {
        static void Main(string[] args)
        {
        Example.GetTime();
        Example.a = 20;
        Console.WriteLine(Example.getIncrement());
        Console.ReadLine();
        }
    }
```

Output:

```
Microsoft Visual Studio Debu

11/1/2025 12:38:53 PM
21


C:\Users\Acer\source\repos\PrivateConstructor\PrivateConstructor\bin\Debug\PrivateConstructor.exe (process 24616) exited
 with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

(e) Implementation of Static Construction in C#

Source Code:

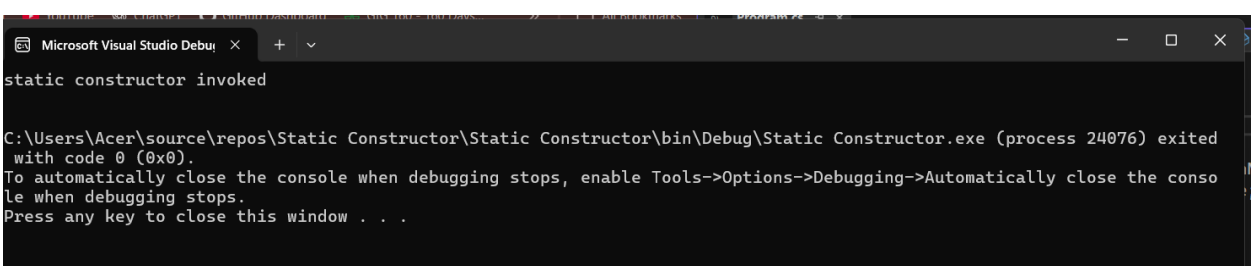
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Static_Constructor
{
    class person
    {
        public static string PersonName;
        public static int PersonAge;

        static person()
        {
            PersonName = "Reeju";
            PersonAge= 23;
            Console.WriteLine("static constructor invoked");
        }
        public static void Getdetails()
        {
            Console.WriteLine("Person Name is:{0}", PersonName);
            Console.WriteLine("Person Age is:{0}",PersonAge);

        }
    }
    internal class Program
    {
        static void Main(string[] args)
        {
            person umar = new person();
            person ali = new person();
            Console.ReadLine();
        }
    }
}
```

Output:



Lab no: 18 **Date:2082/03/13**

Title: (a)Implementation of Static Polymorphism (Compile Time Polymorphism) in C#.

Theory:

- Static Polymorphism (Compile-time Polymorphism)
  Static polymorphism is resolved at compile time. It allows multiple methods in the same class with the same name but different parameters (method overloading) or operators performing different tasks (operator overloading).

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Polymorphism
{
    internal class Program
    {
        public void Add()
        {
            int a = 20;
            int b = 20;
            int c = a + b;
            Console.WriteLine(c);
        }
        public void Add(int a , int b)
        {
            int c = a + b;
            Console.WriteLine(c);
        }
        public void Add (string a , string b)
        {
            string c= a +" "+ b;
            Console.WriteLine(c);
        }
        public void Add (float a , float b)
        {
            float c = a + b;
            Console.WriteLine(c);
        }
        static void Main(string[] args)
        {
            Program p= new Program();
            p.Add();
            p.Add(2.5f, 1.5f);
            p.Add(10, 5);
            p.Add("Reeju" , "Pandit");
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

40
4
15
Reeju Pandit


C:\Users\Acer\source\repos\Polymorphism\Polymorphism\bin\Debug\Polymorphism.exe (process 8016) exited with code 0 (0x0).
Press any key to close this window . . .
```

(b) Implementation of Dynamic Polymorphism (Run Time Polymorphism) in C#.

- Dynamic Polymorphism (Run-time Polymorphism)
  Dynamic polymorphism is resolved at runtime. It is achieved through method overriding, where a derived class provides a specific implementation of a method defined in the base class using virtual and override keywords.

Source Code:

```
using _18b.Dynamic_Polymorphism;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _18b.Dynamic_Polymorphism
{
    class parent
    {
        public virtual void print()
        {
            Console.WriteLine("This is a method of PARENT class");
}
}
class child : parent
{
    public override void print()
    {
        base.print();
        Console.WriteLine("This is a method of CHILD class");
}

}
    internal class Program
    {
        static void Main(string[] args)
        {
            parent p = new child();
            p.print();
            Console.ReadLine();
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

This is a method of PARENT class
This is a method of CHILD class


C:\Users\Acer\source\repos\_18b.Dynamic_Polymorphism\_18b.Dynamic_Polymorphism\bin\Debug\_18b.Dynamic_Polymorphism.exe (
process 18664) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Lab no: 19 **Date:2082/03/16**

Title:(a) Implementation of Synchronous Programming in C#.

Theory:

Synchronous programming means tasks are executed one after another each operation must finish before the next one starts. It is blocking in nature, meaning the program waits for a task to complete before moving forward. This approach is simple and predictable but can make programs slower when handling time-consuming tasks like file I/O or web requests.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace SynchronousProgramming
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Task1();
            Task2();
            Task3();
            Task4();
            Console.ReadLine();

        }
        public static void Task1()
        {
            Console.WriteLine("Task1 Starting....");
            Thread.Sleep(9000);
            Console.WriteLine("Task1 completed");
            Console.WriteLine("Hello MBM Students....");
        }
        public static void Task2()
        {
            Console.WriteLine("Task2 Starting....");
            Thread.Sleep(7000);
            Console.WriteLine("Task2 completed");
        }
        public static void Task3()
        {
            Console.WriteLine("Task3 Starting....");
            Thread.Sleep(8000);
            Console.WriteLine("Task3 completed");
        }
        public static void Task4()
        {
            Console.WriteLine("Task4 Starting....");
            Thread.Sleep(10000);
            Console.WriteLine("Task4 completed");
        }
    }
```

}

Output:

(b) Implementation of Asynchronous Programming in C#.

Theory:

Asynchronous programming allows multiple tasks to run without waiting for each other to complete. It is non-blocking, meaning the program can continue executing other code while waiting for a task (like file read, API call, or database access) to finish. In C#, asynchronous behavior is implemented using the async and await keywords, which improve efficiency and responsiveness—especially in I/O-bound applications.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace AsynchronousProgramming
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Task1();
            Task2();
            Task3();
            Task4();
            Console.ReadLine();
        }

        public static async void Task1()
        {
            await Task.Run(() =>
            {
                Console.WriteLine("Task1 Starting.....");
                Thread.Sleep(40000);
                Console.WriteLine("Task1 Completed....");
            });
        }
```

```
        public static async void Task2()
        {
            await Task.Run(() =>
            {
                Console.WriteLine("Task2 Starting.....");
                Thread.Sleep(20000);
                Console.WriteLine("Task2 Completed....");
            });
        }
        public static async void Task3()
        {
            await Task.Run(() =>
            {
                Console.WriteLine("Task3 Starting.....");
                Thread.Sleep(50000);
                Console.WriteLine("Task3 Completed....");
                Console.WriteLine("Hello World");

            });
        }
        public static async void Task4()
        {
            await Task.Run(() =>
            {
                Console.WriteLine("Task4 Starting.....");
                Thread.Sleep(10000);
                Console.WriteLine("Task4 Completed....");
            });
        }
    }
}
```

Output:

Lab no: 20 **Date:2082/03/17**

Title:(a) Implementation of Exceptional Handling (Logical Implementation) C#.

Theory:

Exception handling in C# is a mechanism to handle runtime errors gracefully without crashing the program. It uses the try, catch, finally, and throw keywords to detect and manage errors logically. This helps maintain the normal flow of execution even when unexpected situations occur.

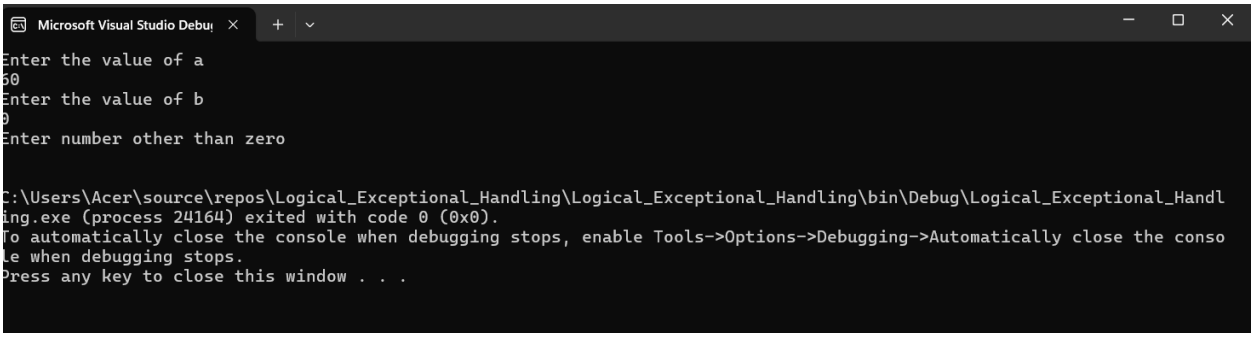
Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Logical_Exceptional_Handling
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int a, b, c;
            Console.WriteLine("Enter the value of a");
            a = int.Parse(Console.ReadLine());
            Console.WriteLine("Enter the value of b");
            b = int.Parse(Console.ReadLine());
            if (b == 0)
            {
                Console.WriteLine("Enter number other than zero");
                Console.ReadLine();
            }
            else
            {
                //b=int.parse(console.Readline());
                c = a / b;
                Console.WriteLine("Division of a by b is=" + c);
                Console.ReadLine() ;
            }
        }
    }

}
```

Output:



b. Implementation of Exception Handling (Try - Catch) in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TryCatch
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                int a, b, c;
                Console.WriteLine("Enter value of a");
                a = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter value of b");
                b = int.Parse(Console.ReadLine());
                c = a / b;
                Console.WriteLine("division of a by b is =" + c);
                Console.ReadLine();
            }

            catch
            {
                Console.WriteLine("Error occured");
                Console.ReadLine();
            }
            finally
            {
                Console.WriteLine("Program Exceuted");
                Console.ReadLine();
            }

        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu
Enter value of a
20
Enter value of b
4
division of a by b is =5

Program Exceuted


C:\Users\Acer\source\repos\TryCatch\TryCatch\bin\Debug\TryCatch.exe (process 24204) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
Microsoft Visual Studio Debu
Enter value of a
20
Enter value of b
0
Error occured

Program Exceuted


C:\Users\Acer\source\repos\TryCatch\TryCatch\bin\Debug\TryCatch.exe (process 24296) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

c) Implementation of Exception Handling (Try – Catch Dividing zero) in C#.

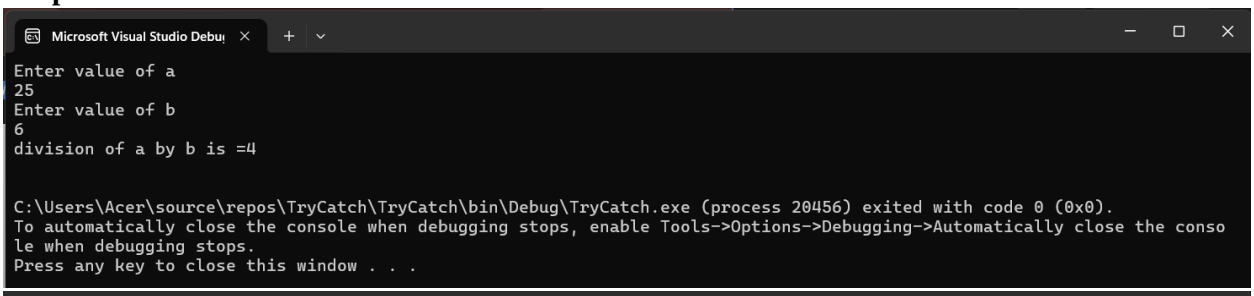
Source Code:

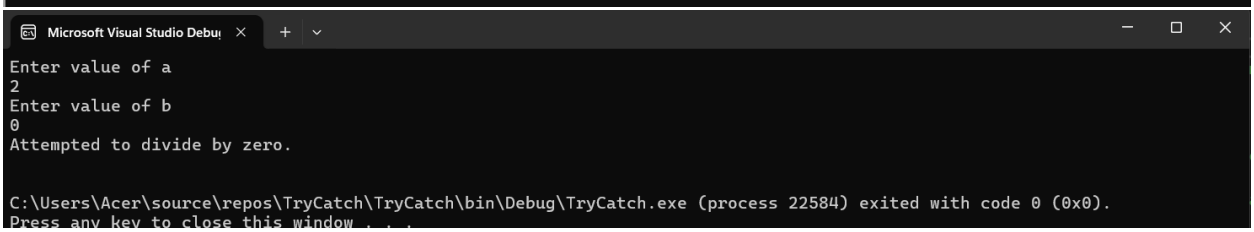
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TryCatch
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                int a, b, c;
                Console.WriteLine("Enter value of a");
                a = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter value of b");
                b = int.Parse(Console.ReadLine());
                c = a / b;
                Console.WriteLine("division of a by b is =" + c);
                Console.ReadLine();
            }
            catch(Exception ex)
            {
                Console.WriteLine(ex.Message);
                Console.ReadLine();
            }
```

```
        }
    }
}
```

Output:





d. Implementation of Exception Handling (Strings) in C#.

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TryCatch
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                int a, b, c;
                Console.WriteLine("Enter value of a");
                a = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter value of b");
                b = int.Parse(Console.ReadLine());
                c = a / b;
                Console.WriteLine("division of a by b is =" + c);
                Console.ReadLine();
            }


            catch (DivideByZeroException ex)
            {
                Console.WriteLine(ex.Message);
                Console.ReadLine();
            }
            catch (FormatException ex)
```

```
            {
                Console.WriteLine(ex.Message);
                Console.ReadLine();
            }
        }
    }
}
```

Output:

```
Microsoft Visual Studio Debu

Enter value of a
56
Enter value of b
3
division of a by b is =18


C:\Users\Acer\source\repos\TryCatch\TryCatch\bin\Debug\TryCatch.exe (process 20528) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
Microsoft Visual Studio Debu

Enter value of a
25
Enter value of b
0
Attempted to divide by zero.


C:\Users\Acer\source\repos\TryCatch\TryCatch\bin\Debug\TryCatch.exe (process 24776) exited with code 0 (0x0).
Press any key to close this window . . .
```